

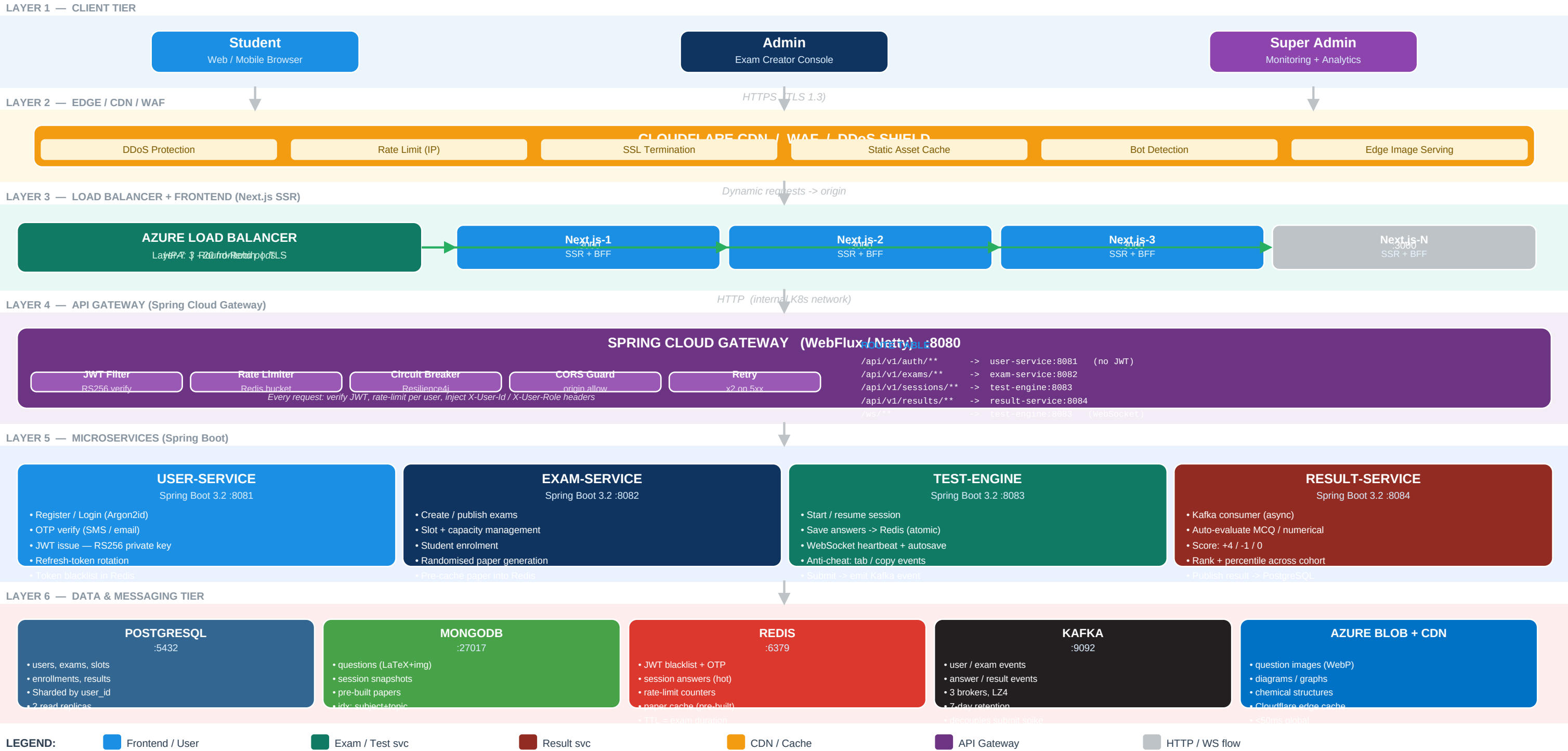
EXAM PLATFORM

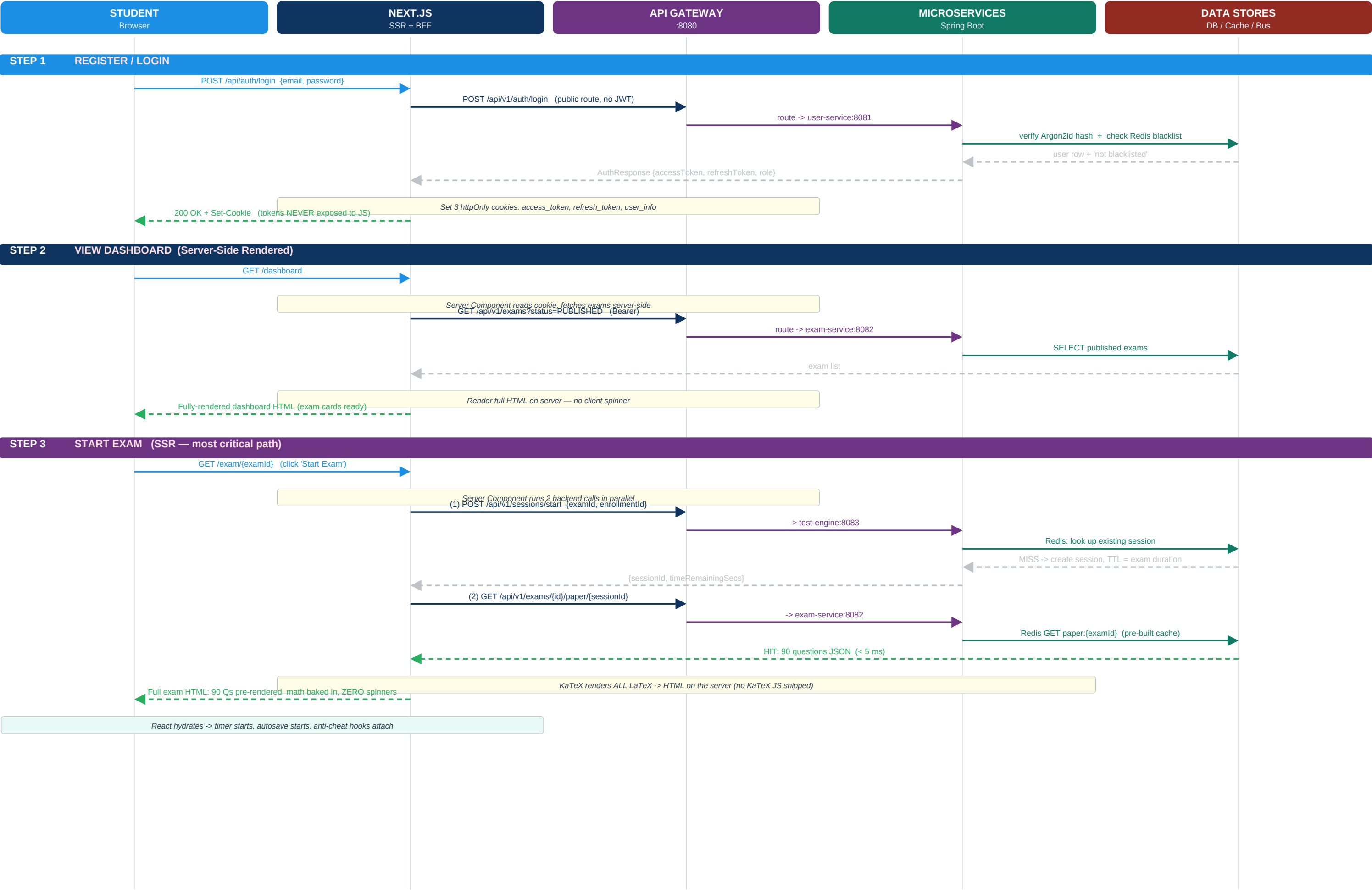
HIGH LEVEL DESIGN (HLD)

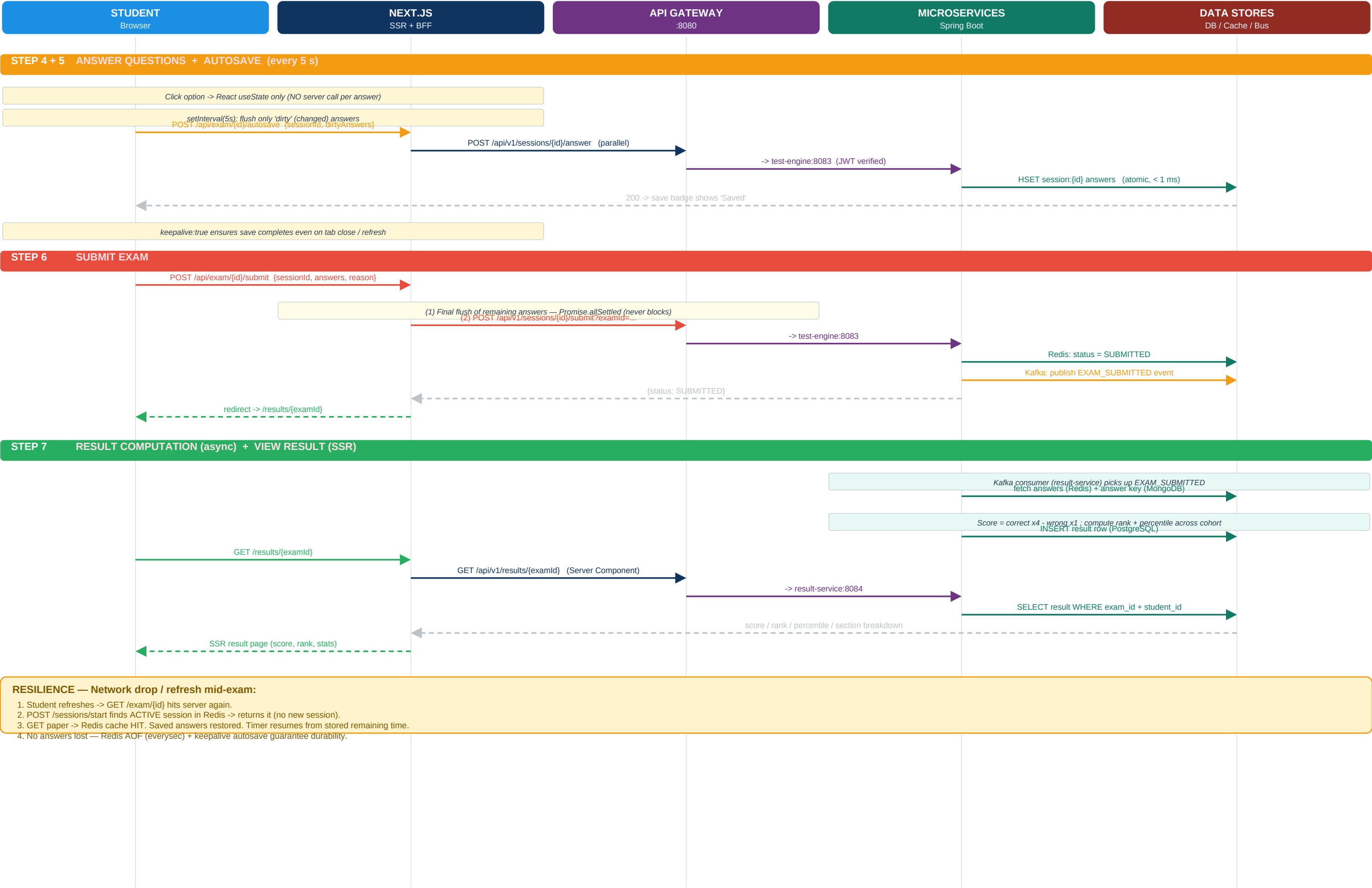
JEE / NEET / CUET / TCS iON Scale | 1M+ Concurrent Users

Version	2.0 (Detailed + UI)
Date	June 2026
Scale	1,000,000+ concurrent users
Availability	99.99% SLA Sub-second latency
Frontend	Next.js 14 (App Router, SSR + BFF)
Backend	Spring Boot 3.2 (4 microservices)
Data	PostgreSQL MongoDB Redis Kafka
Infra	Azure AKS Docker Cloudflare CDN

Microservices • Event-Driven • Server-Side Rendering • Horizontal Scaling • Fault Tolerant







examplatform.com/login

Welcome back

Sign in to your exam account

Email or Phone

you@example.com

Password

Sign In

Don't have an account? [Register here](#)

httpOnly cookie set by Next.js BFF — token is never exposed to JS

LOGIN PAGE — SSG shell + client form

examplatform.com/register

Create Account

Register to take exams

First Name

Last Name

Email

rahul@example.com

Phone (optional)

9876543210

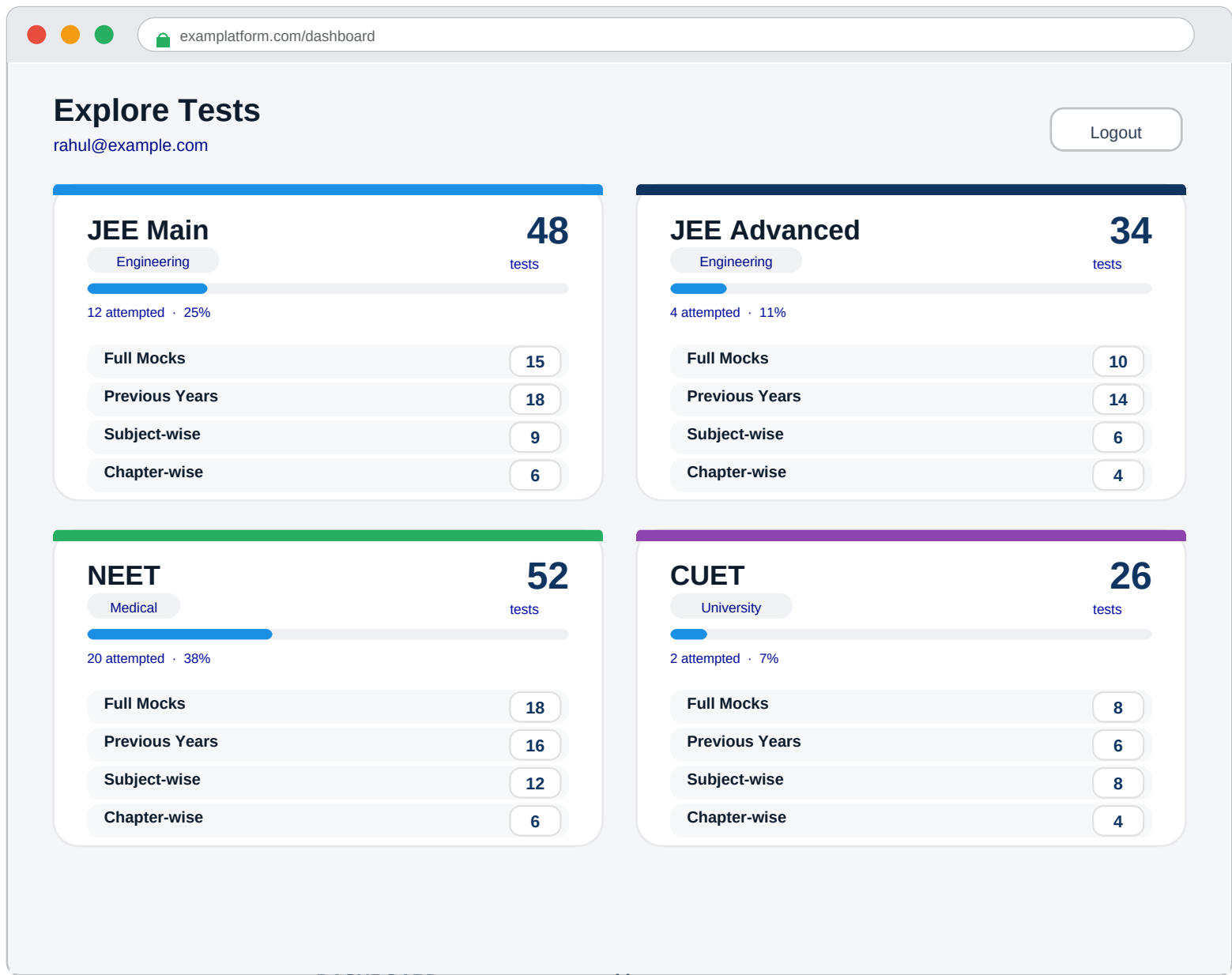
Password

Confirm Password

Create Account

Password rule: 8+ chars, upper, lower, digit, symbol (matches RegisterRequest)

REGISTER PAGE — validates, then auto-login



Category card
 One per exam track:
 JEE Main / Advanced /
 NEET / CUET / BITSAT.

Test-type counts
 Full Mocks, Previous
 Years, Subject-wise,
 Chapter-wise — each
 links to a filtered list.

Progress bar
 attemptedCount / total
 from categories/summary
 endpoint (per student).

Drill-down
 Click a row ->
 /exams?category=..
 &type=.. listing page.

DASHBOARD — exams grouped by category, counts per test type

Pre-rendered server-side:
KaTeX math is HTML, not JS

Timer counts down from
Perlis-stored remaining time
(survives reconnect)

Autosave badge: dirty answers
flushed every 5s (keepalive)

JEE Main 2026 — Full Mock Test 1

⌚ 02:47:13

✓ Saved

! 1 tab switch detected

Question 13 • Mathematics • +4 / -1

Evaluate the following indefinite integral:

$$\int \frac{e^{\arctan x}}{1+x^2} dx =$$

A log(1 + x^2) + c

B log_e [e^{\arctan x}] + c

C e^{\arctan x} + c

D arctan(e^{\arctan x}) + c

Flag

Clear

< Prev

Next >

QUESTIONS

Physics

Chemistry

Maths

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

■ Answered (11)

■ Flagged (4)

□ Not visited (10)

Submit Exam

EXAM INTERFACE — ExamShell (Server Component, renders HTML) + ExamClient ('use client', timer/answers/anti-cheat)

CONFIDENTIAL — Internal Architecture Document | 1M+ Concurrent Users | 99.99% SLA

Page 7

JEE Main 2026 — Full Mock Test 1

Detailed Analysis

284 / 360

78.9% · Rank #1,423 · 97.2 percentile

78 Correct

14 Wrong

8 Skipped

All100

Wrong14

Skipped8

Correct78

Q13

Mathematics

Wrong

-1

Evaluate: $\int \frac{e^{\arctan x}}{(1+x^2)} dx =$

A $\log(1+x^2)+c$

B $\log_e [e^{\arctan x}]+c$

Your answer

C $e^{\arctan x}+c$

Correct answer

D $\arctan(e^{\arctan x})+c$

View Solution

Re-attempt

Q27

Chemistry

Wrong

-1

Which of the following has the highest lattice energy?

A NaCl

B KCl

Your answer

C MgO

Correct answer

D CaO

Hide Solution

Re-attempt

Solution

Lattice energy increases with higher ionic charge and smaller ionic radius (U proportional to q^+q^-/r).
MgO has 2+/2- charges and small radii -> highest lattice energy.

Filter tabs

All / Wrong / Skipped / Correct — jump straight to mistakes.

Colour-coded options

Green = correct answer,
Red = your wrong answer.
Shown per question.

View Solution

Toggles step-by-step solution (KaTeX HTML, pre-rendered server-side).

Re-attempt

Practice the question inline with instant check — does NOT change the recorded score.

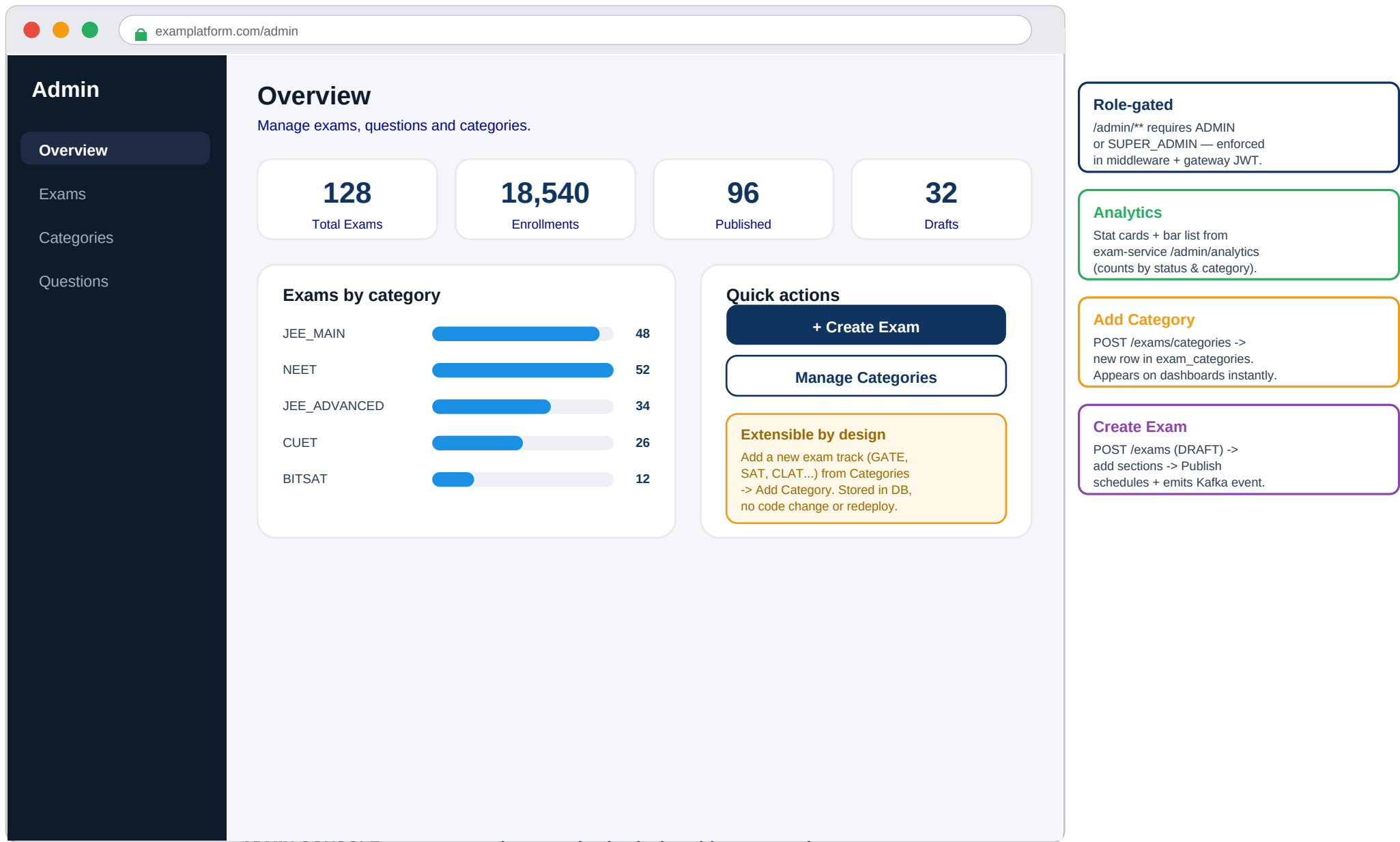
Async pipeline

Detailed result built by result-service Kafka consumer; answer key from MongoDB.

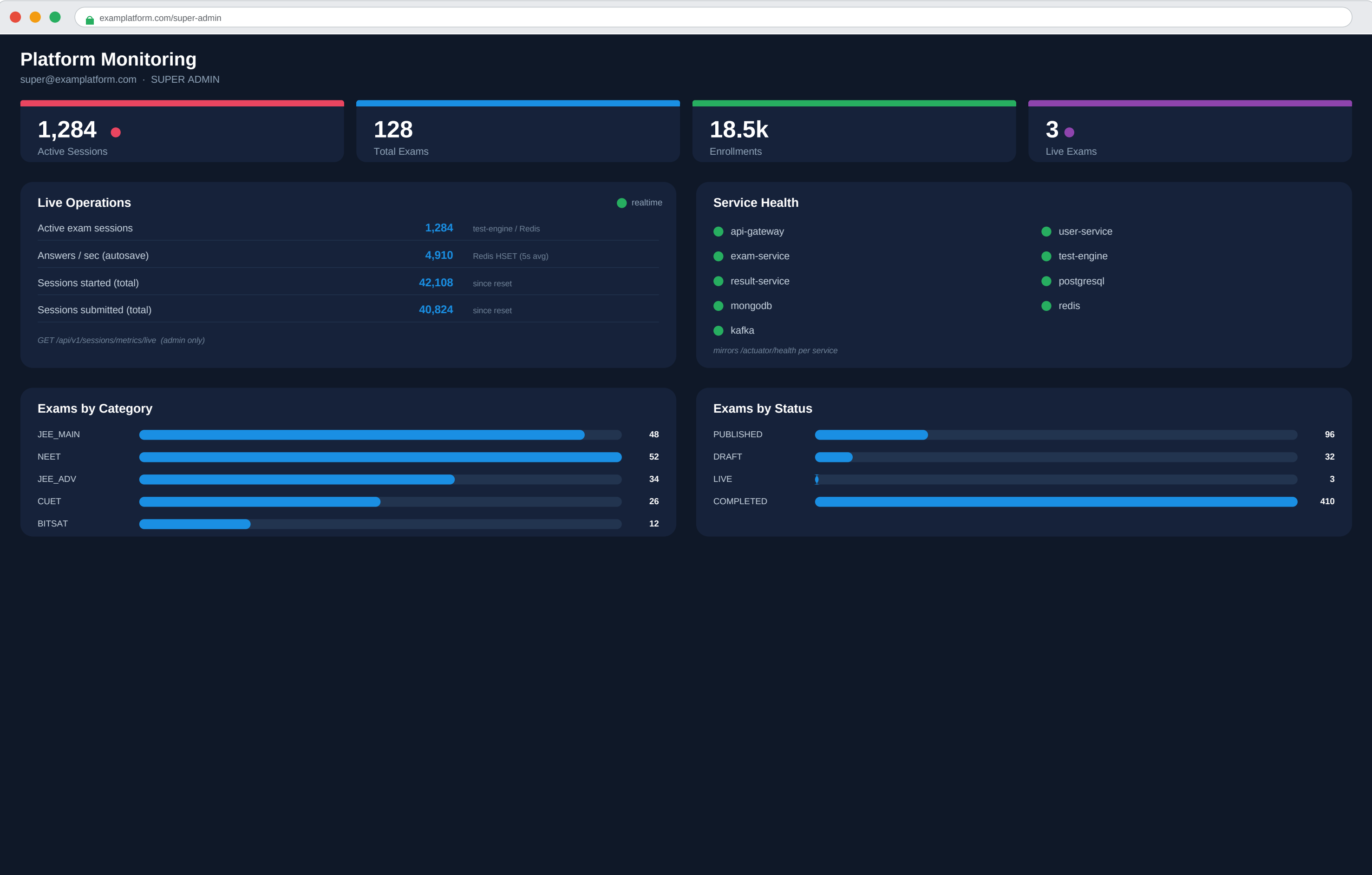
RESULT PAGE — per-question review: your answer vs correct, View Solution, Re-attempt

CONFIDENTIAL — Internal Architecture Document | 1M+ Concurrent Users | 99.99% SLA

Page 8



ADMIN CONSOLE — exam creation, question bank, data-driven categories



Microservice Component Reference

Service	Port	Technology	Responsibilities	Scales
user-service	8081	Spring Boot 3.2 PostgreSQL Redis · Kafka	- Registration & login - OTP (SMS / email) - JWT issue (RS256) - Argon2id hashing - Token blacklist	2 – 10 pods CPU > 70%
exam-service	8082	Spring Boot 3.2 PostgreSQL MongoDB · Redis	- Create / publish exams - Slot management - Enrolment - Paper generation - Redis paper cache	3 – 15 pods CPU > 60%
test-engine	8083	Spring Boot 3.2 MongoDB · Redis Kafka · WebSocket	- Start / resume session - Save answers (Redis) - Heartbeat / autosave - Anti-cheat events - Submit -> Kafka	5 – 50 pods CPU > 50% (hottest)
result-service	8084	Spring Boot 3.2 PostgreSQL MongoDB · Kafka	- Kafka consumer - Auto-evaluate - Score (+4/-1/0) - Rank & percentile - Publish result	2 – 8 pods Kafka lag > 1000
api-gateway	8080	Spring Cloud Gateway (Netty) Redis	- JWT verify (RS256) - Rate limit (Redis) - Circuit breaker - CORS - WS proxy + fallback	3 – 20 pods RPS > 10k
frontend	3000	Next.js 14 App Router TypeScript	- SSR Server Components - BFF API routes - httpOnly cookie JWT - KaTeX server render - Anti-cheat + autosave	3 – 20 pods CPU > 60%

Data Store Reference

Store	Type	Stores	Key design decision
PostgreSQL :5432	Relational ACID	users, exams, slots, enrollments, results	Sharded by user_id range. 2 read replicas serve dashboard & result queries
MongoDB :27017	Document NoSQL	questions (LaTeX+img), snapshots, papers	Index subject+topic+difficulty for fast paper generation. Flexible block scheduler
Redis :6379	In-memory KV	JWT blacklist, OTP, answers, rate-limit, papers	All hot-path data here. TTL = exam duration + 2h. Cluster mode, AOF every 1s
Kafka :9092	Event streaming	user / exam / answer / result event topics	Decouples 500k-submit spike from evaluation. 3 brokers, LZ4, 7-day retention
Azure Blob + CDN	Object + CDN	images, diagrams, chem structures (WebP)	Served from Cloudflare edge (<50ms). WebP ~60% smaller than PNG.

Scaling Strategy & Security Model

Kubernetes HPA — Auto-Scaling

Service	Min	Max	Trigger	Why stateless
frontend	3	20	CPU > 60%	Cookies on client; no server-side session
api-gateway	3	20	RPS > 10,000	Rate-limit state lives in Redis
user-service	2	10	CPU > 70%	DB pooled (HikariCP); no local state
exam-service	3	15	CPU > 60%	Paper in Redis; light DB load
test-engine	5	50	CPU > 50%	Answers in Redis; no DB write per save
result-service	2	8	Kafka lag > 1000	Async consumer; off the hot path
Redis Cluster	3	—	Memory > 70%	Sharded across 3 primary nodes
Kafka	3	—	Partition lag	3 replicas/topic; no loss on broker fail

Security Controls

Threat	Control	Where
JWT theft (XSS)	httpOnly cookies — JS can't read tokens	Next.js login/register routes
CSRF	SameSite=Strict cookies	All Set-Cookie headers
Brute-force login	Redis rate limit: 20 req/s per IP	Gateway · ipKeyResolver
DDoS	Cloudflare WAF + edge rate limiting	Edge layer
Answer scraping	Questions rendered to HTML server-side	ExamShell (Server Component)
Tab switching	visibilitychange -> warn + report	ExamClient.tsx
Copy / paste	copy/cut/paste/contextmenu blocked	ExamClient.tsx
Token replay	JWT blacklisted in Redis on logout	UserService · JwtService
Session hijack	sessionId bound to studentId in Redis	SessionController
Crash / data loss	Redis AOF + Mongo replica + Kafka repl	Infrastructure
Insecure transport	HTTPS TLS 1.3 + HSTS everywhere	Cloudflare · Gateway
Privilege escalation	Role claim in JWT verified at gateway	JwtAuthFilter

Key Design Decisions

Why SSR for the exam page?

Questions are pre-rendered to HTML on the server — never present in the JS bundle, so they can't be scraped from network/DevTools. KaTeX runs on server CPU, so no math-rendering JS ships to the browser and equations appear instantly.

Why Redis for live answers?

1M users saving every 5s ~ 200k writes/sec — beyond PostgreSQL's ~10k/sec. Redis handles 500k+ ops/sec sub-millisecond. After the exam, a Kafka consumer persists final answers to PostgreSQL asynchronously.

Why Kafka between test-engine and result-service?

It decouples the submission spike (500k submits in 60s) from CPU-heavy evaluation. If result-service crashes, Kafka retains and re-delivers messages — no submission is ever lost.

Why httpOnly cookies, not localStorage?

localStorage is readable by any JS (XSS risk). httpOnly cookies are not. With SameSite=Strict and the BFF pattern, the browser never calls Spring Boot directly — preventing both XSS token theft and CSRF.

Why pre-build papers in Redis?

Otherwise 1M students each trigger a MongoDB query + randomisation at start = DB overload. On publish, the paper is built once and cached in Redis with a TTL; every student gets a <5ms cache hit.